INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 07

Automatización de calidad

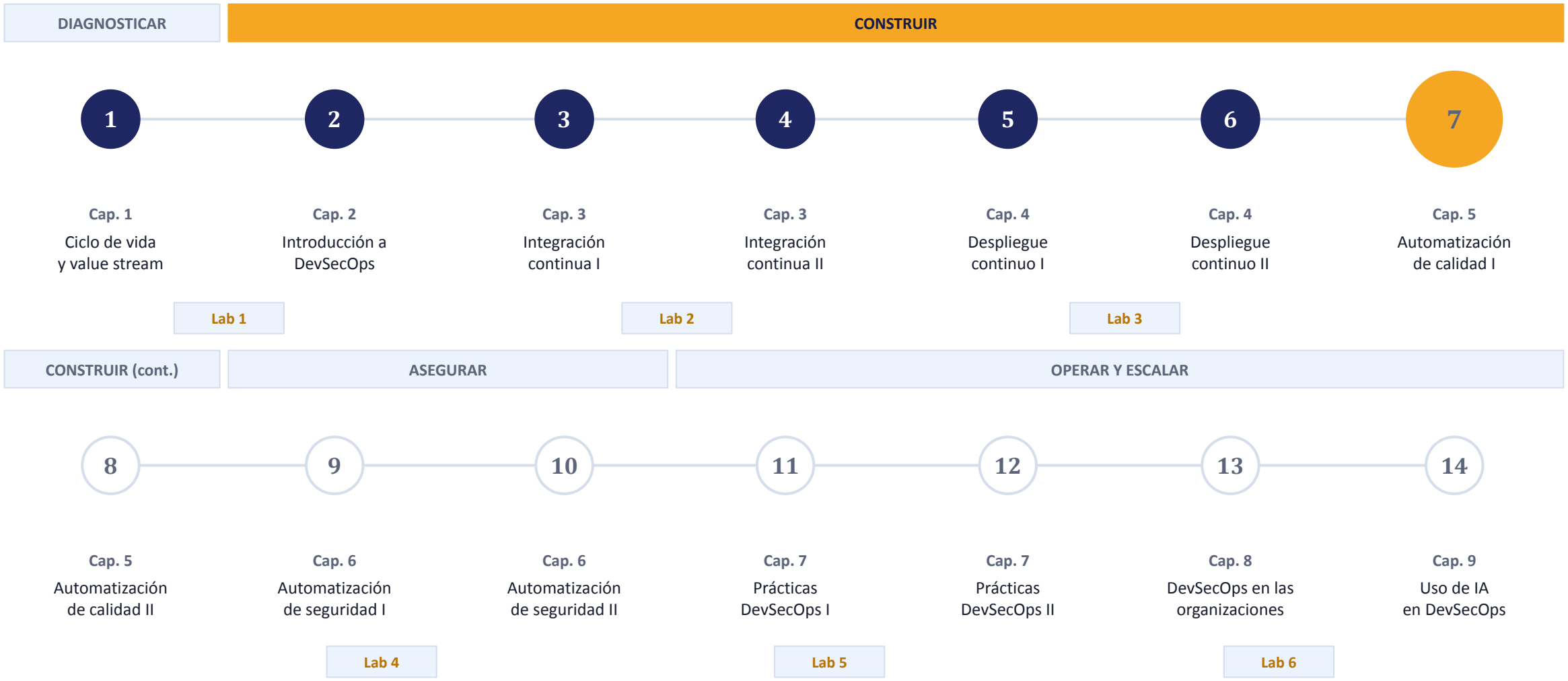
Pirámide de pruebas, pruebas unitarias y pruebas de integración dentro del pipeline

---

Capítulo 5 (1 de 2) · 3 horas | RA2 | Prof. José Felipe del Castillo Salazar

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 7

Objetivo de la sesión y ruta

1. Estrategia de Pruebas
2. Pruebas unitarias en el pipeline
3. Pruebas de integración

Resultado de aprendizaje vinculado: RA2 — pipelines que aseguran calidad, trazabilidad y eficiencia del ciclo de vida.

CONTINUIDAD CON LAS SESIONES 3 A 6

Dónde estamos en el pipeline

Commit	Build	**Pruebas**	Calidad	Seguridad	Publicar	Desplegar	Observar

Lo que ya está resuelto

- El quality gate bloquea la publicación si la cobertura del código nuevo no alcanza el umbral.
- Existe un artefacto único, identificado por su digest, que recorre todos los ambientes.
- El despliegue se verifica con una prueba de humo y una ventana de observación.
- La tasa de fallo de cambios ya tiene una forma concreta de medirse.

Lo que todavía no está decidido

- ¿Qué defecto detecta cada tipo de prueba que los otros no detectan?
- ¿Cuántos minutos de pipeline se justifica gastar en pruebas?
- ¿Qué pruebas corren antes de publicar el artefacto y cuáles después de desplegarlo?
- ¿Qué se hace con una prueba que a veces pasa y a veces falla?

En el Laboratorio 3 el job de pruebas producía la cobertura que leía el quality gate. Esta sesión decide qué debe probar esa etapa y cuánto puede tardar.

A

La forma de la cartera de pruebas

Qué proporción de pruebas va en cada nivel y cuánto tiempo de pipeline consume cada una

Partimos de verificación y validación

Lo que trabajaron en Ingeniería de Software

- Niveles de prueba: unitaria, integración, sistema y aceptación.
- Diseño de casos: particiones de equivalencia y valores límite.
- Verificación y validación como actividades del ciclo de vida.
- Planes de prueba y trazabilidad hacia los requisitos.

Lo que agrega este capítulo

- La prueba como compuerta automática que decide si el artefacto avanza.
- El tiempo de ejecución como restricción de diseño de la suite.
- Determinismo: el mismo commit produce el mismo resultado en cualquier runner.
- La evidencia de cada ejecución queda registrada junto al artefacto.

Una prueba protege el pipeline cuando corre en cada cambio, sin que nadie tenga que decidir ejecutarla.

La pirámide de pruebas

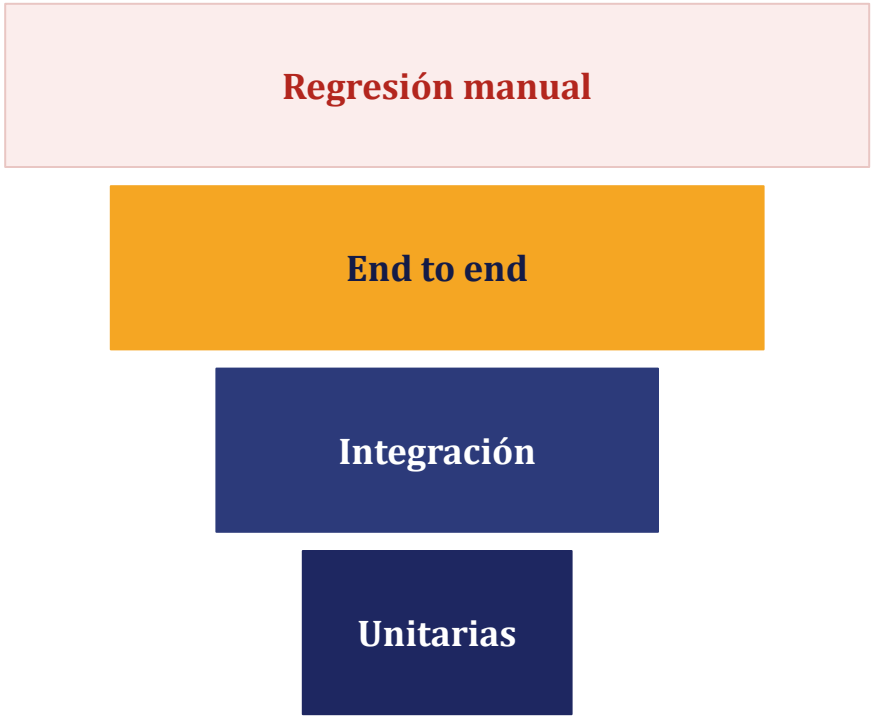


menos pruebas · más lentas · más cerca del usuario

Nivel	Tiempo típico	Qué localiza cuando falla
End to end	minutos	Un recorrido de usuario roto, sin indicar en qué servicio
Integración	segundos	La frontera entre dos componentes: consulta, esquema, serialización
Unitarias	milisegundos	La función exacta y la línea que produce el resultado incorrecto

La proporción se decide por el costo de detectar cada defecto: la base encuentra el error en milisegundos y señala la línea que lo produce.

El triángulo invertido



Malas prácticas

- La regresión se valida a mano antes de cada salida.
- La suite automatizada opera sobre la interfaz y tarda horas.
- Las unitarias existen para alcanzar el umbral de cobertura.
- El equipo vuelve a probar en certificación aunque el pipeline esté en verde.

Impacto

- Probar cuesta tanto que los cambios se acumulan en lotes grandes.
- La certificación pasa a ser la etapa más larga del flujo.
- El retrabajo aparece semanas después, cuando el responsable ya cambió de tarea.

Un equipo con el triángulo invertido reparte la calidad entre las personas que prueban a mano. El pipeline solo confirma que el código compila.

El tiempo de retroalimentación es un presupuesto

Nivel	Cantidad	Tiempo c/u	Total secuencial
Unitarias	2 000	5 ms	10 s
Integración	150	2 s	5 min
End to end	40	45 s	30 min
Suite completa			**≈ 35 min**

Cartera de ejemplo de un servicio mediano. Las cifras son órdenes de magnitud típicos, no mediciones.

1

35 minutos por commit cambian el comportamiento del developer

El desarrollador agrupa cambios para no esperar, y el lote crece.

2

El paralelismo compra tiempo con cómputo

Cuatro ejecutores bajan las end to end a unos 8 minutos.

3

No toda la suite corre en cada commit

Las end to end completas pueden correr antes de promover a qa o producción.

Qué nivel detecta cada defecto

Defecto	Unitaria	Integración	End to end
Regla de negocio mal calculada: descuento, redondeo, tarifa	Detecta	Detecta tarde	Detecta tarde
Consulta SQL incorrecta o migración incompatible	No detecta: el mock lo oculta	Detecta	Detecta tarde
Cambio de formato en la respuesta de una API	Parcial	Detecta	Detecta
Permisos, variables de entorno y configuración del ambiente	No detecta	Parcial	Detecta
Recorrido de usuario roto entre varios servicios/pantallas	No detecta	No detecta	Detecta

Cada nivel se justifica por los defectos que los inferiores no pueden ver.
Una end to end que verifica un cálculo repite en minutos lo que una unitaria resuelve en milisegundos.

B

Pruebas unitarias en el pipeline

Las propiedades que hacen que una prueba unitaria sirva como compuerta

Propiedades de una prueba unitaria confiable

1. **Rápida**
   Se mide en milisegundos. Una suite de miles de pruebas termina en segundos y se ejecuta en cada commit.

2. **Aislada**
   No depende de red, disco, reloj del sistema ni del orden en que corren las demás pruebas.

3. **Repetible**
   El mismo commit produce el mismo resultado en la máquina del desarrollador y en cualquier runner.

4. **Auto-verificable**
   Termina en éxito o fallo por una aserción. Nadie tiene que leer una salida para decidir.

5. **Oportuna**
   Se escribe con el cambio, en el mismo pull request que modifica el comportamiento.

Las cinco propiedades se conocen por su acrónimo en inglés: FIRST.

¿Cómo aislamos las pruebas unitarias?

Stub

Devuelve respuestas fijas para forzar un caso. El servicio de tarifas responde 503 y se verifica el manejo del error.

Mock

Verifica que una interacción ocurrió. Se comprueba que se publicó exactamente un evento de pedido registrado.

Fake

Implementación funcional simplificada. Un repositorio en memoria que guarda y lee pedidos sin base de datos.

Spy

Registra las llamadas sobre el objeto real. Se consulta después cuántas veces se invocó el envío de correo.

1

El “doble” reemplaza lo que el componente (unit) no controla

Red, reloj, base de datos y aleatoriedad. La lógica propia se prueba real.

2

Herramientas útiles en integración con Terceros o recursos limitados

Ejemplo: Servicio Reniec del Estado

Anatomía de una prueba unitaria

SRC/TARIFA.JS · TEST/TARIFA.TEST.JS

```
// src/tarifa.js
function calcularTarifa(pedido, reglas) {        // 1
  const base = pedido.pesoKg * reglas.precioPorKg;
  const recargo = pedido.urgente
    ? base * reglas.recargoUrgente : 0;
  return Math.round((base + recargo) * 100) / 100;
}

// test/tarifa.test.js
describe("calcularTarifa", () => {
  const reglas = { precioPorKg: 4.5, recargoUrgente: 0.2 };

  it("aplica recargo a pedidos urgentes", () => {
    const pedido = { pesoKg: 10, urgente: true };  // 2
    expect(calcularTarifa(pedido, reglas)).toBe(54); // 3
  });

  it("redondea a dos decimales", () => {         // 4
    const pedido = { pesoKg: 3.333, urgente: false };
    expect(calcularTarifa(pedido, reglas)).toBe(15);
  });
});
```

1 **Las reglas entran como parámetro**

La función no lee configuración ni base de datos. No necesita dobles.

2 **Datos mínimos, creados en la prueba**

Todo lo que la prueba usa está a la vista, en el mismo bloque.

3 **Una aserción sobre el resultado**

Se verifica lo observable: el valor que devuelve la función.

4 **Los valores límite se escriben como casos**

El diseño de casos de V&V se convierte en pruebas ejecutables.

Cobertura: qué mide y qué no mide

Qué mide

Qué líneas y qué ramas del código ejecutó la suite durante la corrida.

Qué no mide

Si alguna aserción verificó el resultado de ese código REALMENTE

100% DE LÍNEAS CUBIERTAS · 0 VERIFICACIONES

```
it("calcula la tarifa", () => {
  calcularTarifa(
    { pesoKg: 10, urgente: true },
    reglas,
  );
});
```

1. **El umbral de qality gate permite bloquear Cobertura de Código Total o del incremento de código.**

2. **Las pruebas de mutación miden verificación**
   Cambian un > por un >= y comprueban que alguna prueba falle.

La cobertura es confiable para una afirmación: el código no cubierto no fue ejecutado por ninguna prueba.

C

Pruebas de integración

La frontera con la base de datos, la cola y el servicio externo, levantada dentro del pipeline

Dos alcances de prueba de integración

Integración acotada

Un componente propio contra una dependencia real y efímera: el repositorio contra una base de datos creada para la corrida. Corre en el runner, en segundos.

Integración amplia

Varios servicios desplegados juntos en un ambiente compartido. Requiere que todos estén en la versión correcta al mismo tiempo.

1. **La acotada detecta lo que los mocks ocultan**
   Consultas SQL, migraciones, serialización, tiempos de espera y manejo de conexiones.
2. **La amplia hereda las esperas del ambiente compartido**
   La prueba empieza cuando el ambiente está libre y con las versiones correctas desplegadas.
3. **La compatibilidad entre servicios se valida con validación de Contratos de APIs**

La mayor parte de la integración se prueba acotada, dentro del runner. La amplia queda para lo que solo existe con los servicios desplegados.

Dependencias efímeras dentro del job

GITHUB ACTIONS — BASE DE DATOS COMO SERVICIO DEL JOB

```
jobs:
  integracion:
    runs-on: ubuntu-latest
    services:
      db:                                  # 1
        image: postgres:16
        env:
          POSTGRES_PASSWORD: prueba
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready          # 2
          --health-interval 5s
    env:
      DB_URL: postgres://postgres:prueba@localhost:5432/postgres
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run migrar                # 3
      - run: npm run test:integracion      # 4
```

1 **Nace y muere con el job**

Cada corrida parte de una base vacía. No hay datos de la corrida anterior.

2 **El job espera a que la base responda**

Sin el chequeo de salud, las primeras pruebas fallan por conexión.

3 **Se aplican las migraciones del repositorio**

Una migración incompatible falla aquí, antes de llegar a un ambiente.

4 **Otro comando, otro job, otro tiempo**

Las unitarias no esperan a que la base arranque.

Datos de prueba sin estado compartido

1. **Cada prueba crea los datos que necesita**
   No depende de registros que dejó otra prueba ni de una carga inicial manual.

2. **Identificadores únicos por corrida**
   Dos jobs en paralelo sobre la misma base no chocan por la misma clave.

3. **Limpieza por transacción o por esquema**
   La prueba revierte su transacción al terminar, o cada corrida usa un esquema propio.

4. **Datos sintéticos**
   Una copia de producción anonimizada es la mejor opción, acotada al volumen realmente necesario para pruebas.

Una prueba que depende de datos dejados por otra pasa o falla según el orden de ejecución. Es la causa más frecuente de pruebas de integración inestables.

Puntos clave

1. **La proporción de pruebas se decide por el costo de detectar cada defecto**
   La base de la pirámide localiza en milisegundos; la punta confirma recorridos completos.

2. **El tiempo de retroalimentación es parte del presupuesto del pipeline**
   Una suite que tarda media hora empuja al equipo a acumular cambios.

3. **Una prueba unitaria sirve como compuerta cuando es rápida, aislada y repetible**
   La confianza en el verde se pierde con una sola prueba inestable tolerada.

4. **La cobertura señala el código que ninguna prueba ejecutó**
   La verificación la aportan las aserciones, y las pruebas de mutación la miden.

5. **La integración acotada corre en el runner con dependencias efímeras**
   Detecta consultas, migraciones y serialización antes de que exista un ambiente.

Para la sesión 8: revisen el job de pruebas de su Laboratorio 3 y anoten qué nivel cubre y cuánto tarda.